

Lab 6 - Data Engineering / ML with FHIR

- Due Mar 16 by 8am
- Points 6
- Submitting an external tool
- Available Jan 12 at 8am - Mar 16 at 8am

Lab 6 - Data Engineering / ML with FHIR

Overview

The goal of this lab is to transform data into a usable format for machine learning algorithms. This lab will leverage FHIR as the data format. It will introduce students to various FHIR resources and data engineering techniques to transform data into usable forms needed for machine learning. It will showcase how FHIR can be used as a standard for creating features, as well as using it for inference within a production setting. For the inference portion of the lab, you will be introduced to PyTorch, a common library for machine learning. One of the hardest parts of a machine learning project is understanding the features and transforming the data into a usable format. In addition, FHIR allows for an easier way to extract data from an EMR system that more health systems are adopting.

The goal is to create a features dataset that will be used for machine learning training. This will require transforming FHIR newline-delimited JSON data into new features. This data is from a bulk FHIR export. The dataset will be used to predict if a patient will develop a stroke or not. This lab will leverage Python and the pandas package. pandas is a Python package that is used for data manipulation. In part 2 of the lab, you will use the transformed data and run inference using the pre-trained neural network model and output the probabilities and the predictions.

Learning Objectives

- Use FHIR for machine learning feature engineering.
- Understand how to convert a FHIR bulk export into a tabular format that will work with machine learning algorithms.
- Understand FHIR as a data model, including the structure around resources and to apply common transformations.

- Understand the basics of PyTorch and using deep learning tools for machine learning inference.

Tools Used

- Lab Deployment Repository - <https://github.gatech.edu/cs6440-students/Lab-6-FHIR-Data-Engineering>  [_ \(https://github.gatech.edu/cs6440-students/Lab-6-FHIR-Data-Engineering\)](https://github.gatech.edu/cs6440-students/Lab-6-FHIR-Data-Engineering)
- Python
- pandas Python package: <https://pandas.pydata.org/>  [_ \(https://pandas.pydata.org/\)](https://pandas.pydata.org/)
- PyTorch Python package: <https://pytorch.org/>  [_ \(https://pytorch.org/\)](https://pytorch.org/)
- Docker (optional)

Additional Resources

- **Pandas tutorial video** (<https://gatech.instructure.com/courses/499698/files/69735347>)  [_ \(https://gatech.instructure.com/courses/499698/files/69735347/download?download_frd=1\)](https://gatech.instructure.com/courses/499698/files/69735347/download?download_frd=1)
- **Lab 6 Introduction and Demo** (<https://gatech.instructure.com/courses/499698/pages/lab-6-introduction-and-demo>)
- **One hot encoding**  [_ \(https://deeptai.org/machine-learning-glossary-and-terms/one-hot-encoding\)](https://deeptai.org/machine-learning-glossary-and-terms/one-hot-encoding)
- <https://build.fhir.org/ig/HL7/bulk-data/export.html>  [_ \(https://build.fhir.org/ig/HL7/bulk-data/export.html\)](https://build.fhir.org/ig/HL7/bulk-data/export.html)
- **Standardization (Z-score Normalization)**  [_ \(https://en.wikipedia.org/wiki/Feature_scaling\)](https://en.wikipedia.org/wiki/Feature_scaling)
- **Paper on how FHIR can be used for ML projects**  [_ \(https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7153160/\)](https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7153160/)
- **Sample Pandas Notebook** (<https://gatech.instructure.com/courses/499698/files/69735349?wrap=1>)  [_ \(https://gatech.instructure.com/courses/499698/files/69735349/download?download_frd=1\)](https://gatech.instructure.com/courses/499698/files/69735349/download?download_frd=1)
 - **covid-19-survival-outcomes.csv**
[_ \(https://gatech.instructure.com/courses/499698/files/69735369?wrap=1\)](https://gatech.instructure.com/courses/499698/files/69735369?wrap=1) 
[_ \(https://gatech.instructure.com/courses/499698/files/69735369/download?download_frd=1\)](https://gatech.instructure.com/courses/499698/files/69735369/download?download_frd=1)

Getting Started

Setting Up the Python Environment

This lab is compatible with Python ≥ 3.10 . There are significant changes made in Python 3.10, so if you have an older version installed, you will need to upgrade. (Note for Windows users: The official Python installer does not add itself to the Windows path entirely, so you may want to install through the Windows Store instead.)

For this lab, Python 3.10 or greater is required. Clone or download the required files from the following repo:

<https://github.gatech.edu/cs6440-students/Lab-6-FHIR-Data-Engineering> 

<https://github.gatech.edu/cs6440-students/Lab-6-FHIR-Data-Engineering>

Install the required packages;

```
pip install -r requirements.txt
```

using uv

```
uv sync --all-extras
```

Optional: Setting Up Docker Container

There is also an optional Docker file that can be used that installs the needed Python packages. This can be used if you do not want to install or are having trouble installing Python on your local computer. Open a terminal and navigate into the lab folder here. This is the recommended path as it's fully tested. From the root directory, deploy your Docker instance by running:

```
# Build the image
docker build -t fhir-lab .

# Run the container using mac, linux, or WSL
docker run -it --rm \
  -v $(pwd)/src:/app/src \
  -v $(pwd)/data:/app/data \
  -v $(pwd)/models:/app/models \
  -v $(pwd)/test:/app/test \
  fhir-lab bash
```

Once this is running then you get access to bash on the container where you can then run the pytest commands.

Setting Up Starter Code

The input files are FHIR ndjson files. This is typically the output of a bulk FHIR export. For example, the `patient.ndjson` file has a list of all patient resources that were extracted. This is located in the data folder.

Running Unit Tests

In order to run unit tests locally, you can run pytest located in the `/test` folder. These tests should provide you with a strong indication your code is working. (Gradescope has additional, hidden tests, but none are designed to test edge cases or attempt to confuse you.)

This allows you to test out your code. You can run them individually for each question of the assignment. You can also use these files to see how the transformations are done.

An example command to run in the Docker CLI for test question 3 is: `pytest test/ml_fhir/etl/test_question3.py`.

Running all of the tests at once. You can run the following command.

```
uv run pytest test/ -v
```

(<https://python-poetry.org/docs/#installing-with-the-official-installer>)

Dependencies

Please note that you cannot (and should not need to) add additional dependencies and packages to the project, or it will not work on Gradescope. All code is expected to be very simple, and you are given everything you need. We have already imported the needed packages for you.

Questions and Exercises

For your deliverable for this assignment, the work you submit to Gradescope will all be in the files under the `/src/ml_fhir/etl` folder.

Each question and exercise is fully documented inside the `/src/ml_fhir/etl` files but will be summarized here as well.

PART 1: ETL EXERCISES

Exercise 1

This exercise is found in the file `/src/ml_fhir/etl/patients.py`.

Exercise 1-1

1. Create a function that calculates the age based on the birthday by using the patient resource and a given input date. This will be used to apply to each row of the patient DataFrame to create an `age` column.

Exercise 1-2

1. One hot encode the marital status and add it to the existing dataframe. You can use a built-in pandas functions for this.

There should be 4 columns added for the one hot encoded marital status. It should be an integer 0 or 1.

```
"married_Divorced"  
  
"married_Married"  
  
"married_Widowed"
```

```
"married_Never_Married"
```

Exercise 2

This exercise is found in the file `/src/ml_fhir/etl/observations.py`.

Input: Observations DataFrame

Output: a pandas dataframe grouped by patient id with mean imputed values for target column. The target column has a numerical value of glucose or triglycerides for each patient. You will be using the observation resources that have these values. You will be creating a column that is a numerical value of the observation. Use the following LOINC codes for this exercise.

- The LOINC code for glucose is "2339-0".
- The LOINC code for triglycerides is "2571-8".

You will need to group the observation records by patient id. When using the aggregation function, aggregate by mean. The mean should be rounded to 2 decimal places.

The next step is to impute missing data for the glucose(impute_glucose) and triglycerides(impute_tri) by using the mean of the column. Imputation is important because certain machine learning algorithms only work if there is not any missing data.

Exercise 3

This exercise is found in the file `/src/ml_fhir/etl/observations.py`.

Create a function that will apply mean normalization (z-score normalization) to the glucose and triglycerides column, and round the result to 3 decimal places. (Please note that there are other normalization methods, we are asking for z-score normalization.)

Mean normalization [↗\(https://en.wikipedia.org/wiki/Feature_scaling\)](https://en.wikipedia.org/wiki/Feature_scaling) is used for preparing data for machine learning to help with model convergence, create a uniform scale, and aids in regularization. (Note: Use the standardization formula with the standard deviation)

PART 2: NEURAL NETWORK EXERCISES

Exercise 1

Tensor Conversion (`/src/ml_fhir/inference.py`). Update the function `student_exercise_1_convert_to_tensor`. The goal is to take in the features that are transformed in part 1 and pass them into the neural network as a PyTorch tensor. The tensor should contain only the feature columns needed for the neural network.

You will return a torch tensor `torch.Tensor` with shape `(n_samples, n_features)`

<https://docs.pytorch.org/docs/stable/tensors.html> 

(<https://docs.pytorch.org/docs/stable/tensors.html>)

Exercise 2

Neural Network Inference (`src/ml_fhir/inference.py`). Update the function `student_exercise_2_run_inference`. The goal is to run the pre-trained neural net model. You will return a dictionary with the following structure. Probabilities is the numerical value between 0 and 1 on how likely the event is to occur. Predictions is the binary classification 1 indicating a stroke risk and 0 indicating no stroke risk.

```
{ 'probabilities': np.ndarray,  
  'predictions': np.ndarray  
}
```

Submission

This lab will utilize Gradescope for grading. This assignment will be auto-graded using the Python functions that you have created.

The only code that needs to be updated are the following 3 files:

Part 1

- Question 1: `patients.py`
- Question 2 and 3: `observations.py`

Part 2

- Question 1 and 2 `inference.py`

Upload these files to Gradescope for your submission.

Autograder Results

Results

Code

S1) Test that the get_age (1/1)

S2) Test that the get_marital_status (1/1)

S3) Test that imputation works for glucose (1/1)

S4) Test that imputation works for triglycerides (1/1)

S5) Test mean normilization works (1/1)

S6) Test that the stroke ind is labeled properly (0.5/0.5)

S7) Test that the stroke ind is labeled properly (0.5/0.5)

Model loaded from models/stroke prediction model.pth